Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н. Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н. Э. Баумана)

ФАКУЛЬТЕТ «Информатика, искусственный интеллект и системы управления» (ИУ)
КАФЕДРА «Информационная безопасность» (ИУ8)

Лабораторная работа № 9

ПО КУРСУ

«Алгоритмические языки»

на тему «Многопоточность, TPL, асинхронное программирование»

Студент ИУ8-31 (Группа) Р. Д. Хан (И. О. Фамилия)

Преподаватель Р. Д. Исянов (И.О. Фамилия)

2026 г.

Цель работы

Изучить задачи TPL, потокобезопасную запись результатов и асинхронную работу с сетевыми API.

Условие задачи

1. Загрузить дневные котировки за год для тикеров из ticker.txt с Market Data API. После получения данных запустить Task для расчёта средней цены по формуле: среднее арифметическое дневных (High + Low) / 2. Записать результат в файл в формате Тикер:Цена и обеспечить потокобезопасный доступ.

2. Создать приложение WinForms или WPF: список городов из city.txt и кнопка загрузки погоды выбранного города через API ЛР6. Использовать async/await, чтобы интерфейс не блокировался. Для Linux/macOS допустим консольный вариант с async/await.

Описание решения

Для котировок используется общий класс MarketClient. Даты задаются как сегодня и год назад. Ответ JSON содержит массивы t, h, l, c. HTTP-запрос ожидается через await; по мере получения свечей запускается Task.Run для вычисления среднего и записи. lock защищает общую запись в файл, а Task.WhenAll ожидает завершения всех расчётов. Сетевые запросы идут последовательно с паузой; расчёты и запись выполняются задачами пула потоков. Порядок строк результата не фиксирован.

WinForms-приложение читает названия и координаты городов в ListBox. Асинхронный обработчик кнопки ожидает HTTP-ответ, затем обновляет Label в потоке интерфейса. Кнопка отключается на время операции и включается в finally. Ключ вводится в скрытое поле или берётся из переменной окружения. Дополнительно подготовлен консольный вариант для Linux/macOS.

Для повторяемых проверок предусмотрен режим --demo: три вымышленных тикера и фиксированный JSON погоды. Учебные выходные файлы отделены от реальных. Для полного сбора по исходному ticker.txt нужен пользовательский MARKETDATA_TOKEN; он не был предоставлен. Ошибки отдельных тикеров фиксируются, HTTP 401/403/429 останавливают последующий сбор.

Режим --sample выполняет настоящую загрузку только публичного тикера AAPL без токена и сохраняет averages_sample.txt. Он позволяет проверить сетевой путь отдельно от учебного режима и полного списка.

Порядок запуска

Для котировок задать MARKETDATA_TOKEN, запустить Lab09Stocks. Режим --demo работает без токена на вымышленных данных. Для Windows назначить запускаемым Lab09Weather: выбрать город, ввести ключ и нажать кнопку. Под Linux использовать Lab09WeatherConsole.

Код программы на языке C#

Файл MarketDataCore/Candle.cs

```
namespace MarketDataCore
{
    public class Candle
    {
        public DateTime Date { get; set; }
        public decimal High { get; set; }
        public decimal Low { get; set; }
        public decimal Close { get; set; }
    }
}
```

Файл MarketDataCore/MarketClient.cs

```
using System.Globalization;
using System.Net.Http.Headers;
using System.Text.Json;
namespace MarketDataCore
{
    public class MarketClient : IDisposable
    {
        private HttpClient client = new HttpClient();
        public MarketClient(string token)
        {
            client.Timeout = TimeSpan.FromSeconds(30);
            if (!string.IsNullOrWhiteSpace(token))
                client.DefaultRequestHeaders.Authorization =
                    new AuthenticationHeaderValue("Bearer", token);
        }
        public static List<Candle> Parse(string json)
        {
            using (JsonDocument doc = JsonDocument.Parse(json))
            {
                JsonElement root = doc.RootElement;
                string status = root.GetProperty("s").GetString();
                if (status != "ok")
                    throw new InvalidOperationException("Статус API: " + status);
                JsonElement times = root.GetProperty("t");
                JsonElement highs = root.GetProperty("h");
                JsonElement lows = root.GetProperty("l");
                JsonElement closes = root.GetProperty("c");
                int count = times.GetArrayLength();
                if (count == 0 || highs.GetArrayLength() != count
                    || lows.GetArrayLength() != count || closes.GetArrayLength() != count)
                    throw new InvalidOperationException("Пустые или несогласованные массивы
свечей.");
                List<Candle> result = new List<Candle>();
                for (int i = 0; i < count; i++)
                {
                    Candle candle = new Candle();
                    candle.Date =
DateTimeOffset.FromUnixTimeSeconds(times[i].GetInt64()).UtcDateTime.Date;
                    candle.High = highs[i].GetDecimal();
                    candle.Low = lows[i].GetDecimal();
                    candle.Close = closes[i].GetDecimal();
                    result.Add(candle);
                }
                return result;
            }
        }
        public async Task<List<Candle>> LoadAsync(string ticker, DateTime from, DateTime to)
        {
            string url = "https://api.marketdata.app/v1/stocks/candles/D/"
                + Uri.EscapeDataString(ticker) + "/?from="
                + from.ToString("yyyy-MM-dd", CultureInfo.InvariantCulture)
                + "&to=" + to.ToString("yyyy-MM-dd", CultureInfo.InvariantCulture);
            using (HttpResponseMessage response = await client.GetAsync(url))
            {
                if (!response.IsSuccessStatusCode)
```

```
                    throw new InvalidOperationException("HTTP " + (int)response.StatusCode);
                return Parse(await response.Content.ReadAsStringAsync());
            }
        }
        public static decimal Average(List<Candle> candles)
        {
            if (candles.Count == 0) throw new ArgumentException("Нет свечей.");
            decimal sum = 0;
            foreach (Candle c in candles) sum += (c.High + c.Low) / 2;
            return sum / candles.Count;
        }
        public void Dispose() { client.Dispose(); }
    }
}
```

Файл Lab09Stocks/Program.cs

```
using MarketDataCore;
using System.Globalization;
namespace Lab09Stocks
{
    internal class Program
    {
        private static object fileLock = new object();
        static void Save(string path, string ticker, List<Candle> candles)
        {
            decimal average = MarketClient.Average(candles);
            string line = ticker + ":" + average.ToString("F4", CultureInfo.InvariantCulture);
            lock (fileLock)
            {
                File.AppendAllText(path, line + Environment.NewLine);
                Console.WriteLine(line);
            }
        }
        static async Task Main(string[] args)
        {
            Console.OutputEncoding = System.Text.Encoding.UTF8;
            bool demo = args.Contains("--demo");
            bool sample = !demo && args.Contains("--sample");
            string data = Path.Combine(AppContext.BaseDirectory, "Data");
            string output = demo ? "averages_demo.txt" : sample ? "averages_sample.txt" :
"averages.txt";
            string errors = demo ? "errors_demo.txt" : sample ? "errors_sample.txt" : "errors.txt";
            string[] tickers = sample ? new string[] { "AAPL" } :
File.ReadAllLines(Path.Combine(data,
                demo ? "ticker_demo.txt" : "ticker.txt"))
                .Select(x => x.Trim()).Where(x => x != "").Distinct().ToArray();
            string token = Environment.GetEnvironmentVariable("MARKETDATA_TOKEN");
            if (!demo && !sample && string.IsNullOrWhiteSpace(token))
            {
                Console.WriteLine("Задай переменную MARKETDATA_TOKEN.");
                return;
            }
            File.WriteAllText(output, "");
            File.WriteAllText(errors, "");
            DateTime to = DateTime.Today;
            DateTime from = to.AddYears(-1);
            Console.WriteLine(demo ? "Учебные данные, не реальные котировки."
                : "Период: " + from.ToString("yyyy-MM-dd") + " — " + to.ToString("yyyy-MM-dd"));
            if (sample) Console.WriteLine("Публичный пример API: только AAPL.");
            List<Task> tasks = new List<Task>();
            using (MarketClient client = new MarketClient(token))
            {
                foreach (string ticker in tickers)
                {
                    try
                    {
                        List<Candle> candles;
                        if (demo)
                            candles = MarketClient.Parse(File.ReadAllText(
                                Path.Combine(data, "Demo", ticker + ".json")));
                        else
```

```
                        candles = await client.LoadAsync(ticker, from, to);
                    tasks.Add(Task.Run(() => Save(output, ticker, candles)));
                }
                catch (Exception e)
                {
                    lock (fileLock)
                    {
                        File.AppendAllText(errors, ticker + ":" + e.Message +
Environment.NewLine);
                        Console.WriteLine(ticker + ": ошибка — " + e.Message);
                    }
                    if (e.Message == "HTTP 401" || e.Message == "HTTP 403"
                        || e.Message == "HTTP 429") break;
                }
                if (!demo) await Task.Delay(1100);
            }
            await Task.WhenAll(tasks);
        }
        Console.WriteLine("Сохранено строк: " + File.ReadAllLines(output).Length);
        Console.WriteLine("Результат: " + Path.GetFullPath(output));
        Console.WriteLine("Ошибок: " + File.ReadAllLines(errors).Length);
    }
}
}
```

Файл WeatherCore/City.cs

```
using System.Globalization;
namespace WeatherCore
{
    public class City
    {
        public string Name { get; set; }
        public double Latitude { get; set; }
        public double Longitude { get; set; }
        public override string ToString() { return Name; }
        public static List<City> Load(string path)
        {
            List<City> result = new List<City>();
            foreach (string line in File.ReadAllLines(path))
            {
                if (string.IsNullOrWhiteSpace(line)) continue;
                string[] parts = line.Split('\t');
                if (parts.Length != 2) throw new FormatException("Неверная строка города.");
                string[] coordinates = parts[1].Split(',');
                if (coordinates.Length != 2) throw new FormatException("Неверные координаты.");
                City city = new City();
                city.Name = parts[0].Trim();
                city.Latitude = double.Parse(coordinates[0], CultureInfo.InvariantCulture);
                city.Longitude = double.Parse(coordinates[1], CultureInfo.InvariantCulture);
                if (city.Latitude < -90 || city.Latitude > 90
                    || city.Longitude < -180 || city.Longitude > 180)
                    throw new FormatException("Координаты вне диапазона.");
                result.Add(city);
            }
            return result;
        }
    }
}
```

Файл WeatherCore/WeatherClient.cs

```
using System.Globalization;
using System.Text.Json;
namespace WeatherCore
{
    public class WeatherClient : IDisposable
    {
        private HttpClient client = new HttpClient();
        public WeatherClient() { client.Timeout = TimeSpan.FromSeconds(20); }
        public static string Parse(string json)
```

```
        {
            using (JsonDocument doc = JsonDocument.Parse(json))
            {
                JsonElement root = doc.RootElement;
                double temp = root.GetProperty("main").GetProperty("temp").GetDouble();
                string description = root.GetProperty("weather")[0]
                    .GetProperty("description").GetString();
                return temp.ToString("F2", CultureInfo.InvariantCulture)
                    + " °C, " + description;
            }
        }
        public async Task<string> LoadAsync(City city, string key)
        {
            string url = "https://api.openweathermap.org/data/2.5/weather?lat="
                + city.Latitude.ToString(CultureInfo.InvariantCulture)
                + "&lon=" + city.Longitude.ToString(CultureInfo.InvariantCulture)
                + "&appid=" + Uri.EscapeDataString(key) + "&units=metric&lang=ru";
            using (HttpResponseMessage response = await client.GetAsync(url))
            {
                if (!response.IsSuccessStatusCode)
                    throw new InvalidOperationException("HTTP " + (int)response.StatusCode);
                return Parse(await response.Content.ReadAsStringAsync());
            }
        }
        public void Dispose() { client.Dispose(); }
    }
}
```

Файл Lab09Weather/Program.cs

```
namespace Lab09Weather
{
    internal class Program
    {
        [STAThread]
        static void Main()
        {
            Application.EnableVisualStyles();
            Application.SetCompatibleTextRenderingDefault(false);
            Application.Run(new WeatherForm());
        }
    }
}
```

Файл Lab09Weather/WeatherForm.cs

```
using WeatherCore;
namespace Lab09Weather
{
    public class WeatherForm : Form
    {
        private ListBox cities = new ListBox();
        private Button load = new Button();
        private TextBox key = new TextBox();
        private Label result = new Label();
        private CheckBox demo = new CheckBox();
        public WeatherForm()
        {
            Text = "ЛР9 — Погода";
            ClientSize = new Size(620, 440);
            cities.SetBounds(15, 15, 280, 405);
            key.SetBounds(310, 45, 290, 25);
            key.UseSystemPasswordChar = true;
            key.Text = Environment.GetEnvironmentVariable("OPENWEATHER_API_KEY") ?? "";
            Label caption = new Label();
            caption.Text = "Ключ OpenWeather:";
            caption.SetBounds(310, 20, 280, 20);
            demo.Text = "Учебный ответ JSON";
            demo.SetBounds(310, 85, 290, 25);
            load.Text = "Загрузить погоду";
            load.SetBounds(310, 130, 290, 40);
            result.SetBounds(310, 190, 290, 210);
            Controls.AddRange(new Control[] { cities, key, caption, demo, load, result });
```

```
            load.Click += LoadWeather;
            try
            {
                cities.DataSource = City.Load(Path.Combine(AppContext.BaseDirectory, "Data",
"city.txt"));
            }
            catch (Exception e)
            {
                result.Text = "Ошибка списка городов: " + e.Message;
                load.Enabled = false;
            }
        }
        private async void LoadWeather(object sender, EventArgs e)
        {
            City city = cities.SelectedItem as City;
            if (city == null) return;
            string apiKey = key.Text.Trim();
            if (!demo.Checked && apiKey == "")
            {
                result.Text = "Введи ключ OpenWeather.";
                return;
            }
            bool useDemo = demo.Checked;
            load.Enabled = false;
            result.Text = "Загрузка: " + city.Name;
            try
            {
                string weather;
                if (useDemo)
                {
                    string path = Path.Combine(AppContext.BaseDirectory, "Data",
"weather_demo.json");
                    weather = WeatherClient.Parse(await File.ReadAllTextAsync(path));
                }
                else
                {
                    using (WeatherClient client = new WeatherClient())
                        weather = await client.LoadAsync(city, apiKey);
                }
                result.Text = city.Name + Environment.NewLine + weather;
                if (useDemo) result.Text += Environment.NewLine + "Учебные данные.";
            }
            catch (Exception error) { result.Text = "Ошибка: " + error.Message; }
            finally { load.Enabled = true; }
        }
    }
}
```

Файл Lab09WeatherConsole/Program.cs

```
using WeatherCore;
namespace Lab09WeatherConsole
{
    internal class Program
    {
        static async Task Main(string[] args)
        {
            Console.OutputEncoding = System.Text.Encoding.UTF8;
            try
            {
                string data = Path.Combine(AppContext.BaseDirectory, "Data");
                List<City> cities = City.Load(Path.Combine(data, "city.txt"));
                for (int i = 0; i < cities.Count; i++)
                    Console.WriteLine((i + 1) + ". " + cities[i].Name);
                Console.Write("Номер города: ");
                if (!int.TryParse(Console.ReadLine(), out int index)
                    || index < 1 || index > cities.Count)
                {
                    Console.WriteLine("Неверный номер.");
                    return;
                }
                City city = cities[index - 1];
```

```
                string weather;
                if (args.Contains("--demo"))
                {
                    Console.WriteLine("Учебный ответ JSON, не текущая погода.");
                    weather = WeatherClient.Parse(File.ReadAllText(Path.Combine(data,
"weather_demo.json")));
                }
                else
                {
                    string key = Environment.GetEnvironmentVariable("OPENWEATHER_API_KEY");
                    if (string.IsNullOrWhiteSpace(key))
                    {
                        Console.WriteLine("Задай OPENWEATHER_API_KEY.");
                        return;
                    }
                    using (WeatherClient client = new WeatherClient())
                        weather = await client.LoadAsync(city, key);
                }
                Console.WriteLine(city.Name + ": " + weather);
            }
            catch (Exception e) { Console.WriteLine("Ошибка: " + e.Message); }
        }
    }
}
```

Результаты выполнения программы

В настоящем API-запуске AAPL за 2025-11-01—2026-11-01 получены 250 свечей; средняя цена — 291,1553. На учебных данных проверены средние 11, 19 и 30. Все исходники скомпилированы, включая WinForms. На Linux разобраны 192 города и проверен консольный модуль погоды; окно Windows и живая погода не запускались. Для всех тикеров нужен токен.

Lab09Stocks_sample_run.txt

```
Период: 2025-11-01 — 2026-11-01
Публичный пример API: только AAPL.
AAPL:291.1553
Сохранено строк: 1
Результат: /workspace/scratch/6823d4720338/compile_work/averages_sample.txt
Ошибок: 0
```

Lab09Stocks_run.txt

```
Учебные данные, не реальные котировки.
TEST_SAME:30.0000
TEST_DOWN:19.0000
TEST_UP:11.0000
Сохранено строк: 3
Результат: /workspace/scratch/6823d4720338/compile_work/averages_demo.txt
Ошибок: 0
```

Фрагмент фактического вывода консольного варианта погоды

```
192. Яунде
Номер города: Учебный ответ JSON, не текущая погода.
Москва: 12.50 °C, учебный пример
```

Выводы по работе

Проверены задачи TPL, защищённая запись и обработка настоящих свечей AAPL. Реализована асинхронная загрузка погоды для WinForms; ключ вводится при запуске.